

Отчет по РК2 по ПиК ЯП. Вариант РК1 4Б

Задание

Рубежный контроль представляет собой разработку тестов на языке Python.

- 1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.
- 2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

Листинг кода (в конце есть с красивым оформлением)

```
from dataclasses import dataclass
from itertools import groupby
from operator import itemgetter
import unittest
```

```
@dataclass
```

```
class Computer:
```

```
    """
```

```
    Класс для объекта "Компьютер".
```

```
    """
```

```
    id: int
```

```
    name: str
```

```
    uptime: float
```

```
    smart_classroom_id: int
```

```
@dataclass
```

```
class SmartClassroom:
```

```
    """
```

```
    Класс для объекта "Дисплейный класс" в понимании комнаты с
    компьютерами.
```

```
    """
```

```
    id: int
```

```
    name: str
```

```
@dataclass
```

```
class ComputerAtSmartClassroom:
```

```
    """
```

```
    Класс для связи многие-ко-многим.
```

```
    """
```

```
    id: int
```

```
    computer_id: int
```

```
    smart_classroom_id: int
```

```
# Вспомогательные функции для осмысленного получения данных из  
# объединенной таблицы
```

```
def computer_name(x): return x[0].name
```

```
def computer_uptime(x): return x[0].uptime
```

```
def classroom_name(x): return x[1].name
```

```
def sort_computers(table: list[Computer]):
```

```
    return sorted(table, key=lambda x: (computer_name(x), classroom_name(x)))
```

```
def count_computers(table: list[Computer]):
```

```
    return sorted(map(lambda x: (x[0], len(list(x[1]))), groupby(
```

```
        sorted(table, key=classroom_name), key=classroom_name)),
```

```
    key=itemgetter(1))
```

```
def filter_computers(table: list[Computer], key: str):
```

```
    return sorted(filter(lambda x: computer_name(x).endswith(key), table),
```

```
    key=lambda x: (computer_name(x), classroom_name(x)))
```

```

def main():
    computers = [
        Computer(0, "Windows 11 PC Intel", 3400.18, 0),
        Computer(1, "Windows 11 PC Intel", 1459.76, 1),
        Computer(2, "Ubuntu 22 PC AMD", 5637.89, 2),
        Computer(3, "Windows 10 PC AMD", 2164.54, 0),
        Computer(4, "Debian New PC AMD", 10637.55, 1),
        Computer(5, "ArchLinux PC AMD", 3448.26, 2),
        Computer(6, "Manjaro PC Intel", 460.34, 2)
    ]

    smart_classrooms = [
        SmartClassroom(0, "Windows room"),
        SmartClassroom(1, "Mixed room"),
        SmartClassroom(2, "Linux room")
    ]

    # Связь многие-ко-многим
    computers_at_smart_classrooms = [
        # Уже существующие связи
        ComputerAtSmartClassroom(0, 0, 0),
        ComputerAtSmartClassroom(1, 1, 1),
        ComputerAtSmartClassroom(2, 2, 2),
        ComputerAtSmartClassroom(3, 3, 0),
        ComputerAtSmartClassroom(4, 4, 1),
        ComputerAtSmartClassroom(5, 5, 2),
        ComputerAtSmartClassroom(6, 6, 2),

        # Новые связи. Добавили Windows и Linux ПК к их "приватным"
        # комнатам

```

```
ComputerAtSmartClassroom(7, 1, 0),  
ComputerAtSmartClassroom(8, 4, 2)  
]
```

Объединенная таблица для связи один-ко-многим. Считаем 1 раз вместо двух и сохраняем

```
combined_table = list(map(lambda x: (x, list(filter(  
    lambda y: y.id == x.smart_classroom_id, smart_classrooms)))[0]), computers))
```

```
print('Запрос Б-1')
```

```
print(*list(map(lambda x: ',\t'.join([computer_name(x), classroom_name(x), str(  
    computer_uptime(x)) + 's']), sort_computers(combined_table))), sep='\n')  
print('\n')
```

```
print('Запрос Б-2')
```

```
print(*list(map(lambda x: f'{x[0]}:\t{x[1]} items',  
    count_computers(combined_table))), sep='\n')  
print('\n')
```

Создадим объединенную таблицу в таком же формате, как и до этого

```
combined_table.clear()  
  
combined_table = list(map(lambda x: (list(filter(lambda y: y.id ==  
x.computer_id, computers))[0], list(  
    filter(lambda y: y.id == x.smart_classroom_id, smart_classrooms))[0]),  
computers_at_smart_classrooms))
```

Используется окончание 'AMD'

```
print('Запрос Б-3')
```

```
print(*list(map(lambda x: f'{computer_name(x)},\t{classroom_name(x)}',  
    filter_computers(combined_table, 'AMD'))), sep='\n')
```

```
print('\n')
```

```
class TestDB(unittest.TestCase):
```

```
    computers = [  
        Computer(0, "Windows 11 PC Intel", 3400.18, 0),  
        Computer(1, "Windows 10 PC AMD", 1459.76, 0),  
        Computer(2, "Ubuntu 22 PC AMD", 5637.89, 1)  
    ]
```

```
    smart_classrooms = [  
        SmartClassroom(0, "Windows room"),  
        SmartClassroom(1, "Mixed room")  
    ]
```

```
    def test_sort(self):
```

```
        combined_table = [  
            (self.computers[0], self.smart_classrooms[0]),  
            (self.computers[1], self.smart_classrooms[0]),  
            (self.computers[2], self.smart_classrooms[1])  
        ]
```

```
        sorted_computers = sort_computers(combined_table)
```

```
        for i in range(len(sorted_computers) - 1):
```

```
            self.assertGreaterEqual(computer_name(  
                sorted_computers[i + 1]), computer_name(sorted_computers[i]))
```

```
            if computer_name(sorted_computers[i + 1]) ==  
computer_name(sorted_computers[i]):
```

```
self.assertGreaterEqual(classroom_name(
    sorted_computers[i + 1]), classroom_name(sorted_computers[i]))
```

```
def test_count_by_classrooms(self):
```

```
    combined_table = [
        (self.computers[0], self.smart_classrooms[0]),
        (self.computers[1], self.smart_classrooms[0]),
        (self.computers[2], self.smart_classrooms[1])
    ]
```

```
    counted_computers = count_computers(combined_table)
```

```
    self.assertEqual(counted_computers[0][1], 1)
```

```
    self.assertEqual(counted_computers[1][1], 2)
```

```
def test_filter(self):
```

```
    combined_table = [
        (self.computers[0], self.smart_classrooms[0]),
        (self.computers[1], self.smart_classrooms[0]),
        (self.computers[0], self.smart_classrooms[1]),
        (self.computers[1], self.smart_classrooms[1]),
        (self.computers[2], self.smart_classrooms[1])
    ]
```

```
    keyword = 'AMD'
```

```
    filtered_computers = filter_computers(combined_table, keyword)
```

```
    for record in filtered_computers:
```

```
        self.assertTrue(computer_name(record).endswith(keyword))
```

```
if __name__ == '__main__':  
    # main()  
    unittest.main()
```


Результат выполнения (в конце есть в виде скриншота)

...

Ran 3 tests in 0.000s

OK

Листинг кода в стиле Codesnap

```
from dataclasses import dataclass
from itertools import groupby
from operator import itemgetter
import unittest

@dataclass
class Computer:
    """
    Класс для объекта "Компьютер".
    """
    id: int
    name: str
    uptime: float
    smart_classroom_id: int

@dataclass
class SmartClassroom:
    """
    Класс для объекта "Дисплейный класс" в понимании
    комнаты с компьютерами.
    """
    id: int
    name: str

@dataclass
class ComputerAtSmartClassroom:
    """
    Класс для связи многие-ко-многим.
    """
    id: int
    computer_id: int
    smart_classroom_id: int
```

```
# Вспомогательные функции для осмысленного получения
данных из объединенной таблицы
def computer_name(x): return x[0].name
def computer_uptime(x): return x[0].uptime
def classroom_name(x): return x[1].name

def sort_computers(table: list[Computer]):
    return sorted(table, key=lambda x:
(computer_name(x), classroom_name(x)))

def count_computers(table: list[Computer]):
    return sorted(map(lambda x: (x[0], len(list(x[1]))),
groupby(
    sorted(table, key=classroom_name),
key=classroom_name)), key=itemgetter(1))

def filter_computers(table: list[Computer], key: str):
    return sorted(filter(lambda x:
computer_name(x).endswith(key), table), key=lambda x:
(computer_name(x), classroom_name(x)))

def main():
    computers = [
        Computer(0, "Windows 11 PC Intel", 3400.18, 0),
        Computer(1, "Windows 11 PC Intel", 1459.76, 1),
        Computer(2, "Ubuntu 22 PC AMD", 5637.89, 2),
        Computer(3, "Windows 10 PC AMD", 2164.54, 0),
        Computer(4, "Debian New PC AMD", 10637.55, 1),
        Computer(5, "ArchLinux PC AMD", 3448.26, 2),
        Computer(6, "Manjaro PC Intel", 460.34, 2)
    ]
```

```

smart_classrooms = [
    SmartClassroom(0, "Windows room"),
    SmartClassroom(1, "Mixed room"),
    SmartClassroom(2, "Linux room")
]

# Связь многие-ко-многим
computers_at_smart_classrooms = [
    # Уже существующие связи
    ComputerAtSmartClassroom(0, 0, 0),
    ComputerAtSmartClassroom(1, 1, 1),
    ComputerAtSmartClassroom(2, 2, 2),
    ComputerAtSmartClassroom(3, 3, 0),
    ComputerAtSmartClassroom(4, 4, 1),
    ComputerAtSmartClassroom(5, 5, 2),
    ComputerAtSmartClassroom(6, 6, 2),
    # Новые связи. Добавили Windows и Linux ПК к их
    "приватным" комнатам
    ComputerAtSmartClassroom(7, 1, 0),
    ComputerAtSmartClassroom(8, 4, 2)
]

# Объединенная таблица для связи один-ко-многим.
Считаем 1 раз вместо двух и сохраняем
combined_table = list(map(lambda x: (x, list(filter(
    lambda y: y.id == x.smart_classroom_id,
smart_classrooms))[0])), computers))

print('Запрос Б-1')
print(*list(map(lambda x:
',\t'.join([computer_name(x), classroom_name(x), str(
    computer_uptime(x)) + 's']),
sort_computers(combined_table))), sep='\n')
print('\n')

```

```

print('Запрос Б-2')
print(*list(map(lambda x: f'{x[0]}:\t{x[1]} items',
                 count_computers(combined_table))), sep='\n')
print('\n')

# Создадим объединенную таблицу в таком же формате,
как и до этого
combined_table.clear()
combined_table = list(map(lambda x:
(list(filter(lambda y: y.id == x.computer_id,
computers))[0], list(
    filter(lambda y: y.id == x.smart_classroom_id,
smart_classrooms))[0]), computers_at_smart_classrooms))

# Используется окончание 'AMD'
print('Запрос Б-3')
print(*list(map(lambda x:
f'{computer_name(x)}:\t{classroom_name(x)}',
    filter_computers(combined_table, 'AMD'))),
sep='\n')
print('\n')

class TestDB(unittest.TestCase):
    computers = [
        Computer(0, "Windows 11 PC Intel", 3400.18, 0),
        Computer(1, "Windows 10 PC AMD", 1459.76, 0),
        Computer(2, "Ubuntu 22 PC AMD", 5637.89, 1)
    ]

    smart_classrooms = [
        SmartClassroom(0, "Windows room"),
        SmartClassroom(1, "Mixed room")
    ]

    def test_sort(self):

```

```

        combined_table = [
            (self.computers[0],
self.smart_classrooms[0]),
            (self.computers[1],
self.smart_classrooms[0]),
            (self.computers[2],
self.smart_classrooms[1])
        ]

        sorted_computers =
sort_computers(combined_table)

        for i in range(len(sorted_computers) - 1):
            self.assertGreaterEqual(computer_name(
                sorted_computers[i + 1]),
computer_name(sorted_computers[i]))
            if computer_name(sorted_computers[i + 1]) ==
computer_name(sorted_computers[i]):
                self.assertGreaterEqual(classroom_name(
                    sorted_computers[i + 1]),
classroom_name(sorted_computers[i]))

        def test_count_by_classrooms(self):
            combined_table = [
                (self.computers[0],
self.smart_classrooms[0]),
                (self.computers[1],
self.smart_classrooms[0]),
                (self.computers[2],
self.smart_classrooms[1])
            ]

            counted_computers =
count_computers(combined_table)

```

```

        self.assertEqual(counted_computers[0][1], 1)
        self.assertEqual(counted_computers[1][1], 2)

    def test_filter(self):
        combined_table = [
            (self.computers[0],
self.smart_classrooms[0]),
            (self.computers[1],
self.smart_classrooms[0]),
            (self.computers[0],
self.smart_classrooms[1]),
            (self.computers[1],
self.smart_classrooms[1]),
            (self.computers[2],
self.smart_classrooms[1])
        ]

        keyword = 'AMD'
        filtered_computers =
filter_computers(combined_table, keyword)

        for record in filtered_computers:
            self.assertTrue(computer_name(record).endswi
th(keyword))

if __name__ == '__main__':
    # main()
    unittest.main()

```

Результат выполнения в виде скриншота

```
• (venv) [arrgn@DESKTOP-JDUQ1EN PCPL_IU5-32B]$ python ./main.py
...
-----
Ran 3 tests in 0.000s

OK
◦ (venv) [arrgn@DESKTOP-JDUQ1EN PCPL_IU5-32B]$
```